CMSC 33100 Project: Multi-tiered Filesystem Policies

Kathir Meyyappan, Johnston Liu

University of Chicago — CMSC 33100: Advanced Operating Systems

Source code: https://github.com/kathirmeyyappan/filestore-tiering

---

1. Abstract

Storage systems deal with a tradeoff between cost and speed, forcing most deployments to maintain a mix of fast and slow media. Existing tiering solutions operate at the block level or assume network-attached or cloud storage, leaving a gap for lightweight, filesystem-level tiering on single machines. We present a modular tiering daemon that uses symlinks to maintain a size-limited hot directory, transparently promoting and demoting files across tiers based on pluggable eviction policies. We implement and evaluate seven such policies—spanning classical (LRU, LFU), adaptive (LRU-2Q, ARC, LeCaR, CACHEUS), and learned (decision tree) approaches—across eight synthetic workload profiles designed to isolate recency, frequency, scan resistance, and phase-shift behavior. Our benchmarks show that no single policy dominates across workloads; performance is highly workload-dependent, underscoring the need for workload characterization when selecting a tiering strategy. Nonetheless, demonstration that we can use tiering policies to circumvent a hard storage limit in such systems is useful.

---

2. Introduction & Motivation

2.1 The Cost–Speed Tradeoff

Storage cost increases as latency decreases. This leads to most systems maintaining less low-latency, high speed storage. In filesystems, this is typically not a problem as “disk” latency is already far higher than in-memory storage. Nonetheless, there are certainly instances when it is advantageous to be conscious of where files live. For example, on single machines, if we have limited “tier 1” disk storage, but the entire working directory must live on this disk (i.e. the user, for convenience, should only be aware of this disk), there are no good solutions to deferring files to cheaper storage on the same machine.

2.2 Existing Approaches and Our Solution

Cloud-native solutions like AWS EFS and distributed sharding handle tiering well but assume network access. Block-level approaches (e.g. ZFS tiering, dm-cache) operate *below* the filesystem, coupling policy decisions to the OS's block layer and adding overhead. They do not target the case of a single machine with multiple local storage tiers and no network dependency. This leaves a gap for a lightweight, filesystem-level tiering mechanism that operates entirely within userspace, which we have implemented.

2.3 Use Cases

This could possibly (with more fleshing out) come in handy in environments that are highly concerned with security, field-deployed systems, air-gapped workstations, cost-sensitive local workloads, etc.

2.4 Research Questions

Obviously, there is some level of stretching happening when we claim that such stringent requirements are realistic, and furthermore, can only be solved with our proposed file tiering system. But regardless, it is interesting to frame the following central questions:

- How efficient can single-machine filesystem-level symlinking be?
- Do policies from other domains (page replacement, scheduling, caching) transfer well to file-tier management?

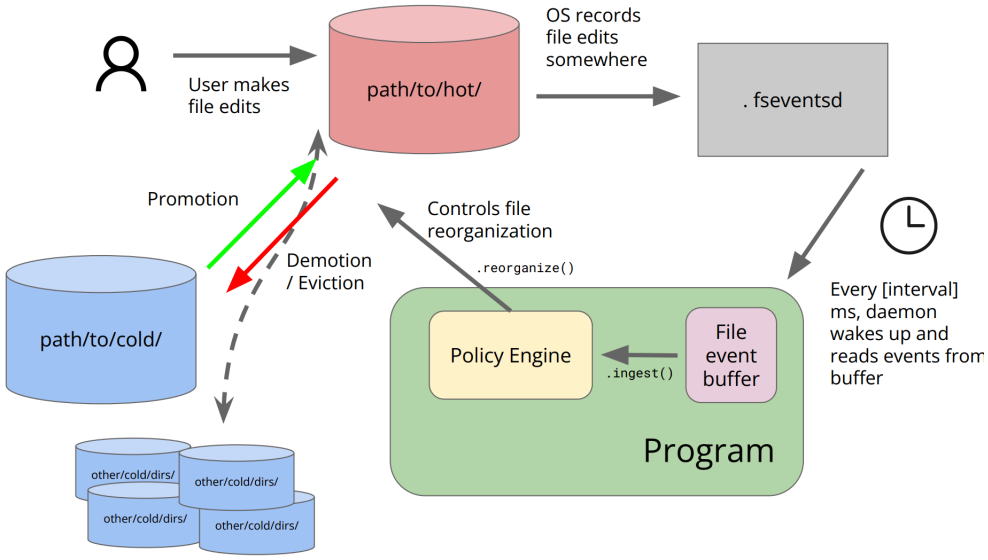
---

3. Design

3.1 Goal & Constraints

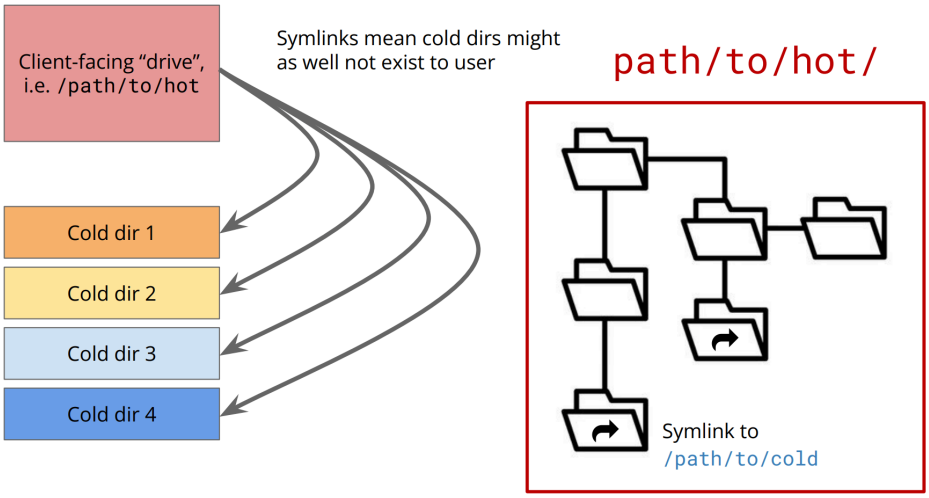
Frequently accessed files' contents are stored in a "hot" drive. The hot drive also stores the root references to all files in a user's working model. Stale or infrequently accessed files' *contents* are stored in a "cold" drive (in a generalized model, we have many levels of cold drives). A policy engine determines how file contents move between these layers. It bases decisions on some facets of user interaction with the filesystem, and ideally keeps file movement low while maintaining a high hit rate in warmer directories.

3.2 High-Level Architecture



The user makes file edits in `path/to/hot/`. The OS records these edits via `.fseventsd` (in MacOS—this is handled differently for other OS but will still work). Every `[interval]` ms, the daemon wakes up and reads events from a file event buffer. The buffer is drained into the `PolicyEngine` via `.ingest()`. The engine then calls `.reorganize()`, which controls file reorganization — promoting files to `path/to/hot/` or demoting/evicting files to `path/to/cold/` and other cold directories.

3.3 Storage Hierarchy



The `path/to/hot/` directory is the client-facing tier — the user only interacts with this directory. When files are "demoted", their file contents are moved to `path/to/cold/` or other cold directories, while a symlink is left behind in `path/to/hot/` so the user's view of the filesystem remains unchanged. In a generalized model, there can be many levels of cold directories (as shown in the architecture diagram above). Promotion moves file contents back into `path/to/hot/`; demotion/eviction moves them deeper into the cold tiers in a generalized model.

3.4 Key Components

- **PolicyEngine** — Modular policies with internal data structures to make reorganization decisions. Uses the `TierState` interface to move files and record stats. New policies are easily added — just inherit from `PolicyEngine`, implement methods, and drop in.
- **Watcher (Daemon)** — `main()` runs a daemon thread that periodically wakes and drains a buffer that has been pulling from the file edits pool (FSEvents for MacOS).
- **Benchmarking Engine** — Benching engine, non-intrusive logging modules, etc.

Please see our full implementation with detailed writeups on these (and other) structures at https://github.com/kathirmeyyappan/filestore-tiering.

---

4. Policies

We implemented a couple policies, some of which are simple and ubiquitous in many fields of computing, and others of which are specific to thread scheduling, OS page eviction, machine learning, etc. It should be noted that while we implemented these to the best of our ability, gaps in implementation robustness may have surfaced because of the complex nature of our actual `TierState` management and `PolicyEngine` hooks.

For all policies, the general trigger for eviction is the same. In the context of our system, we have a fixed storage ceiling (in bytes) for the hot directory (and colder directories in more complex implementations), where files reside at first. When that total space threshold is crossed, we "evict" files (in accordance with the policy) by moving their contents to a colder directory while keeping a canonical symlink in the hot directory. Promotions, where file contents are moved from the cold directory back into the hot directory, also occur based on the policy's specifications.

4.1 LRU

LRU is a simple, common policy. When a file is created or edited, it is moved to the hot directory and also marked as the most recently accessed file. The same goes for touched files that are already in the hot directory. When eviction is triggered, we move the least recently touched files' contents to the cold directory until we are back under the space threshold for the hot directory.

4.2 LFU

This is identical to LRU in how we promote, but we evict not by recency, but by which files in the hot directory have been accessed the least since their promotion.

4.3 LRU-2Q

LRU-2Q (Johnson & Shasha, 1994) fixes a weakness of plain LRU where sequential scans over many unique files flush the entire hot directory. It splits the hot tier into two queues: a small FIFO buffer called A1in (25% of hot capacity) and a larger LRU queue called Am (75%). New files enter A1in. If a file is re-accessed after being evicted from A1in, it is promoted directly into Am, since two accesses across an eviction boundary suggest genuine demand. Files that are never re-accessed simply pass through A1in and are evicted to cold without disturbing Am. A ghost list (A1out) tracks the keys of A1in evictions to enable this "second chance" lookup..

4.4 ARC (Adaptive Replacement Cache)

ARC (Megiddo & Modha, FAST '03) keeps two LRU queues in the hot tier. T1 holds files accessed once (recency) and T2 holds files accessed more than once (frequency). Ghost lists B1 and B2 track evictions from each. An adaptive parameter p controls the capacity split between T1 and T2. When a file evicted from T1 is re-accessed (a B1 ghost hit), p increases, giving recency more room. When a T2 eviction is re-accessed (a B2 ghost hit), p decreases, favoring frequency. This lets ARC automatically tune itself to the workload, unlike LRU-2Q's fixed split.

4.5 LeCaR

LeCaR (Vietri et al., HotStorage '18) maintains two eviction experts, one using LRU and one using LFU, and picks between them via a weighted coin flip at eviction time. Evicted files enter a ghost list tagged with which expert chose them. When a ghost-listed file is re-accessed, that expert is penalized and weight shifts toward the other, with recent mistakes penalized more heavily. Over time, this regret minimization steers the policy toward whichever strategy better fits the current workload.

4.6 CACHEUS

CACHEUS (Gil et al., FAST '21) extends LeCaR with three improvements. Its LRU expert is replaced with SR-LRU (Scan-Resistant LRU), where new files enter a probationary FIFO and only on a second access are promoted into a protected LRU queue, so one-hit scan files are evicted cheaply without polluting the protected set. Its LFU expert is replaced with CR-LFU (Cache-Resident LFU), where frequency counts decay exponentially over time, preventing stale-popular files from permanently blocking new ones. Finally, the learning rate adapts with the formula `lr = max(0.001, lr_init * |2w - 1|)`, learning slowly when uncertain and quickly when committed to one expert.

4.7 Decision Tree

The decision tree policy (inspired by LRB; Song et al., NSDI '20) takes a learned approach to eviction. Each file in the hot directory is scored on four features: recency, frequency, file size, and average inter-access time. A decision tree regressor is trained to predict which files will not be re-accessed, i.e. which evictions will be regret-free. A ghost list provides training labels: if an evicted file is re-accessed (a ghost hit), the eviction was a mistake; if it expires from the ghost list without being re-accessed, the eviction was correct. The tree is periodically retrained on these samples, allowing the policy to adapt as workload patterns shift.

5. Benchmarking Methodology

5.1 Metrics

We recorded the following metrics: cache hit rate (%), bytes promoted, bytes demoted, `.ingest()` latency, and `.reorganize()` latency.

5.2 Workload Generation

We used seeded (deterministic) operations — creates, edits, and deletes — and synchronously ran policies on file modification workloads.

5.3 A Note on Throughput

Raw throughput is not a valid comparison metric here because synchronous reorganization adds wall-clock overhead that would not exist in production, where the daemon runs asynchronously. A "do nothing" policy trivially maximizes throughput but achieves a 0% hit rate. Placement quality is what matters.

That having been said, it might have been nice to have as a secondary metric to hitrate and whatnot as it synergizes in long-running jobs. Alas, there were some implementation troubles in getting this working so we have decided to forgo this metric.

---

6. Workload Presets

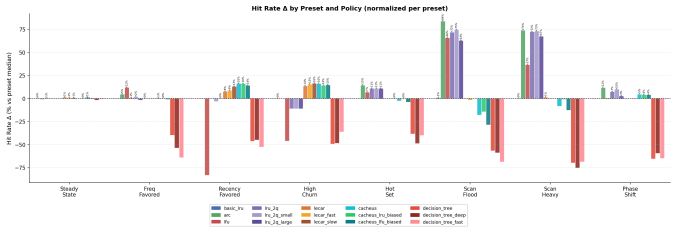
We defined eight workload presets with fixed parameters to isolate specific policy strengths and weaknesses.

Preset	Description	Expected to Reward
`steady_state`	Uniform random access, no exploitable pattern	Baseline — all policies similar
`frequency_favored`	Edits concentrated on small set of old files	Frequency-aware policies
`recency_favored`	High creation rate, edits on newest files	Recency-aware policies
`high_churn`	Heavy create+delete, shifting working set	Adaptation speed
`hot_set`	Moderate frequency skew (skew=3.0)	Milder frequency reward
`scan_flood`	Many one-time creates + stable frequent core	Scan resistance
`scan_heavy`	Extreme scan_flood variant	FIFO probation (LRU-2Q)
`phase_shift`	Alternates frequency- and recency-dominant phases	Adaptation speed

---

7. Results

7.1 Hit Rate Across Policies and Workloads

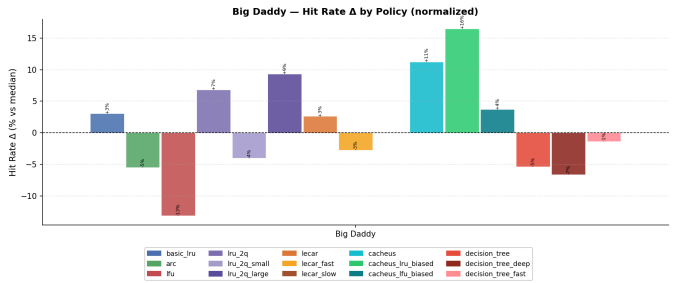


The hit rate delta chart shows each policy's deviation from the per-preset median, normalizing across workloads so relative differences stand out. On `steady_state`, all policies sit near zero since uniform random access offers no exploitable

pattern (~20.5% hit rate across the board). On `frequency_favored`, LFU leads at 74.1% since it directly optimizes for access frequency, with ARC second at 69.2% and the remaining classical policies in the 65-68% range. The decision tree variants trail at 24-40%, suggesting the learned model struggles to capture strong frequency skew within its retraining window.

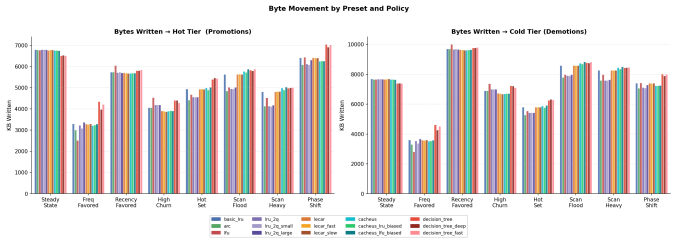
On `recency_favored`, all policies achieve low hit rates (1-7%) because the 40% create rate constantly introduces new files that overwhelm the hot directory. CACHEUS edges slightly ahead at ~7%, while LFU drops to 1% since frequency counts are useless when files are mostly brand-new. On the scan workloads (`scan_flood` and `scan_heavy`), ARC and LRU-2Q pull ahead at 25-32%, validating that their admission filtering protects the hot set from one-time scan traffic. Basic LRU drops to 14-18% since scans flush its entire queue. On `phase_shift`, ARC leads at 32.6% thanks to its adaptive p parameter, with LRU-2Q close behind at 31.3%.

7.2 Mixed Workload Comparison



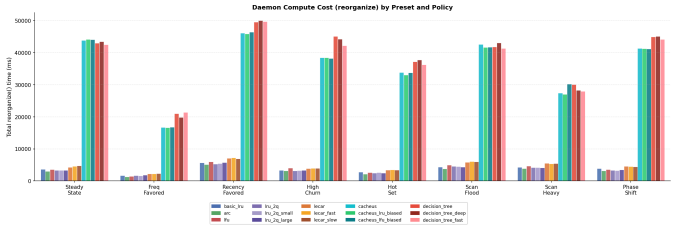
Across all eight workloads, no single policy dominates. ARC is the most consistently strong performer, finishing first or second on five of eight workloads (`frequency_favored`, `hot_set`, `scan_flood`, `scan_heavy`, `phase_shift`). LFU wins decisively on `frequency_favored` but collapses on `recency_favored` and `high_churn`. LRU-2Q is competitive with ARC on scan-heavy workloads but falls behind on workloads that require dynamic rebalancing. LeCaR and CACHEUS perform well on `high_churn` (where rapid adaptation matters) but do not outperform ARC on scan or phase-shift workloads. The decision tree variants consistently underperform, likely due to insufficient training data within the benchmark window and the cost of periodic retraining.

7.3 Byte Movement (Promotion & Demotion Rates)



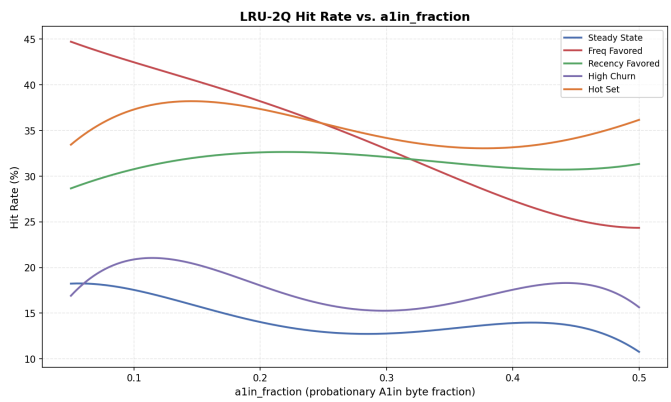
Byte movement correlates inversely with hit rate: policies with higher hit rates move fewer bytes because they make better placement decisions. On `frequency_favored`, ARC writes ~3,300 KB to cold versus ~3,600 KB for basic LRU, a ~9% reduction. On `steady_state`, all policies move roughly the same amount (~7,800 KB cold writes) since no policy can outperform random on a uniform workload. The decision tree variants show elevated byte movement on most workloads, reflecting their lower hit rates and the churn from poor eviction choices. The efficiency scatter (hit rate vs. cold-write cost) makes this tradeoff clear: ARC and LRU-2Q land in the upper-left (high hit rate, low byte cost), while decision tree variants cluster in the lower-right.

7.4 Daemon Latency (.ingest() and .reorganize())



Ingest latency is low and similar across all policies (roughly 1-1.5 seconds total across 400 poll cycles), since `.ingest()` is primarily event filtering and bookkeeping. Reorganize latency splits into two tiers. The classical policies (basic LRU, LFU, ARC, LRU-2Q) and LeCaR complete `.reorganize()` in 1-7 seconds total depending on workload. CACHEUS and decision tree are roughly 10x more expensive, taking 16-50 seconds total. For CACHEUS, this comes from maintaining SR-LRU and CR-LFU with exponential decay. For the decision tree, periodic retraining with smartcore's regressor is the dominant cost. In production, `.reorganize()` runs asynchronously so it would not block file access, but the compute overhead is still relevant at scale.

7.5 Tunable Parameter Progression



We swept LRU-2Q's `a1in_fraction` parameter from 0.05 to 0.50 across five workloads. The original 2Q paper (Johnson & Shasha, 1994) claims that the full algorithm with the A1out ghost list "eliminates tuning to a massive extent," since even if a file is evicted from A1in prematurely, the ghost list catches it on re-access and promotes it to Am anyway. Our results only partially confirm this. Of the five workloads, only `recency_favored` is genuinely insensitive, with hit rate varying by just 4.4 percentage points across the full sweep range. `hot_set` shows moderate sensitivity (6.3pp), while `steady_state` (11.3pp), `high_churn` (8.6pp), and `frequency_favored` (20.8pp) all exhibit meaningful variation. On `frequency_favored`, a smaller A1in (0.05-0.10) nearly doubles hit rate compared to 0.50, since maximizing Am capacity directly retains the stable working set under strong frequency skew. The ghost list reduces but does not eliminate this sensitivity: it provides a second chance for prematurely evicted files, but when Am is too small the protected queue simply cannot hold enough of the hot working set, and ghost-driven promotions cannot compensate. These results suggest that the paper's claim holds best for workloads without strong frequency or locality structure, and that practitioners should still tune `a1in_fraction` when the workload exhibits skew.

8. Discussion

8.1 No Clear Winner

The central takeaway is that no single policy dominates across workloads. ARC is the most broadly competitive, but LFU beats it handily on `frequency_favored`, and LeCaR/CACHEUS edge it out on `high_churn`. The performance gap is also highly workload-dependent: on `steady_state` all policies are within 1 percentage point of each other, while on `frequency_favored`

the gap between LFU (74%) and the decision tree (24%) is 50 points. This confirms that workload characterization is essential when selecting a tiering strategy.

8.2 Policy Transferability

Classical caching and page replacement policies (LRU, LFU, LRU-2Q, ARC) transfer reasonably well to filesystem tiering. The core abstractions of recency, frequency, capacity limits, and eviction map directly onto our hot/cold model. The main friction point is granularity: these policies were designed for fixed-size pages or cache lines, while files vary in size. Our implementation handles this by tracking byte budgets rather than entry counts, but the mismatch means a single large file eviction can free space for many small promotions, creating bursty behavior that page-level policies do not anticipate. The learned approach (decision tree) transferred least well, likely because our benchmark generates relatively short traces (20,000 operations) that do not give the regressor enough signal to generalize.

8.3 Practical Viability

The system works as intended: all files remain accessible via their original paths in the hot directory, with symlinks transparently pointing to cold storage when contents have been demoted. The daemon overhead for classical policies is low enough for production use. The main practical limitation is that our benchmarks use synthetic workloads on a single physical disk, so we do not observe the latency differential that would exist between an NVMe hot tier and a spinning disk cold tier. In a real deployment, the hit rate differences we observe would translate directly into user-perceived latency improvements.

---

9. Future Work

Several extensions would strengthen this system. Multi-tiering (N > 2 active tiers) would allow graduated demotion rather than a binary hot/cold split. Physical drive separation would introduce realistic latency differentials and make throughput a meaningful secondary metric. Time-series logging of adaptive parameters (ARC's p, LeCaR's w, CACHEUS's learning rate) would allow deeper analysis of convergence behavior. The decision tree policy could benefit from longer training windows, richer features (e.g. file type, directory locality), or a switch to online learning rather than periodic batch retraining. Finally, testing on real-world file access traces rather than synthetic workloads would validate whether our findings generalize.

---

10. Conclusion

We presented a modular filesystem tiering daemon that uses symlinks to maintain a size-limited hot directory, transparently promoting and demoting files across tiers based on pluggable eviction policies. We implemented and evaluated seven policies spanning classical (LRU, LFU), adaptive (LRU-2Q, ARC, LeCaR, CACHEUS), and learned (decision tree) approaches across eight synthetic workload profiles. Our benchmarks confirm that no single policy dominates: ARC is the most broadly competitive, but specialized policies (LFU for frequency-heavy, LeCaR for high-churn) win on their target workloads. Performance is highly workload-dependent, underscoring the need for workload characterization when selecting a tiering strategy. The system itself is lightweight, modular, and practically viable for single-machine deployments where all file references must remain in a single directory with a hard storage limit.

References

1. Gil, T., Park, J., Choi, J., and Oh, S. "CACHEUS: Adaptive Learning Cache Management." Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST '21), 2021.
2. Johnson, T. and Shasha, D. "2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm." Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), 1994, pp. 439-450.
3. Megiddo, N. and Modha, D. S. "ARC: A Self-Tuning, Low Overhead Replacement Cache." Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST '03), 2003, pp. 115-130.

4. Song, Z., Berger, D. S., Li, K., and Lloyd, W. "Learning Relaxed Belady for Content Distribution Network Caching." Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20), 2020, pp. 607-621.
5. Vietri, G., Rodriguez, L. V., Martinez, W. A., Lyons, S., Liu, J., Rangaswami, R., Zhao, M., and Narasimhan, G. "Driving Cache Replacement with ML-based LeCaR." Proceedings of the 10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage '18), 2018.

Appendix

A. Source Code

The full implementation is available at github.com/kathirmeyyappan/filestore-tiering. Key modules:

Component	Path	Description
Policy trait	`src/policy_engine.rs`	`PolicyEngine` trait defining `ingest()`, `reorganize()`, and `pick_for_demotion()`
Daemon loop	`src/daemon.rs`	Main poll loop, policy instantiation via `make_policy()`
Tier filesystem	`src/tier_fs.rs`	`move_to_tier()`: atomic file moves with symlink replacement
Tier state	`src/tier_state.rs`	`TierState`: tracks which files live in which tier and current byte usage
Basic LRU	`src/policies/basic_lru.rs`	Single LRU queue, evicts least-recently-accessed file
LFU	`src/policies/lfu.rs`	Frequency-count eviction with min-heap
LRU-2Q	`src/policies/lru_2q.rs`	Full 2Q with A1in FIFO, A1out ghost list, and Am LRU
ARC	`src/policies/arc.rs`	Adaptive Replacement Cache with T1/T2 + B1/B2 ghost lists
LeCaR	`src/policies/lecar.rs`	Regret-minimization blend of LRU and LFU
CACHEUS	`src/policies/cacheus.rs`	SR-LRU + CR-LFU with adaptive learning rate
Decision tree	`src/policies/decision_tree.rs`	Periodic batch retraining via smartcore regressor
Benchmark harness	`src/bench/workload.rs`	Trace generation, replay, and metric collection
Benchmark presets	`benches/presets.rs`	All 8 workload configurations and CLI driver

B. Workload Preset Parameters

All presets share a base configuration: `warmup_ops=3000`, `measure_ops=20000`, `poll_interval_ops=50`, `depth=3`, `file_size=256-768 B`. The table below shows per-preset overrides.

Preset	`hot_cap`	`create%`	`delete%`	`edit%`	`skew`	Design intent
`steady_state`	50,000	10	5	85	1.0	Uniform baseline; no exploitable pattern
`frequency_favored`	40,000	3	0	97	5.0	Stable hot set; frequency signal dominates
`recency_favored`	40,000	40	0	60	0.3	New-file storm; recency signal dominates
`high_churn`	50,000	30	20	50	0.3	Rapidly shifting live set

Preset	`hot_cap`	`create%`	`delete%`	`edit%`	`skew`	Design intent
`hot_set`	50,000	10	5	85	3.0	Moderate skew on older files
`scan_flood`	40,000	30	5	65	3.0	One-time scan traffic + stable core
`scan_heavy`	25,000	35	5	60	4.0	Extreme scan pressure
`phase_shift`	50,000	10	0	90	4.0/0.3	Alternating freq/recency phases (4 phases, 25% each)